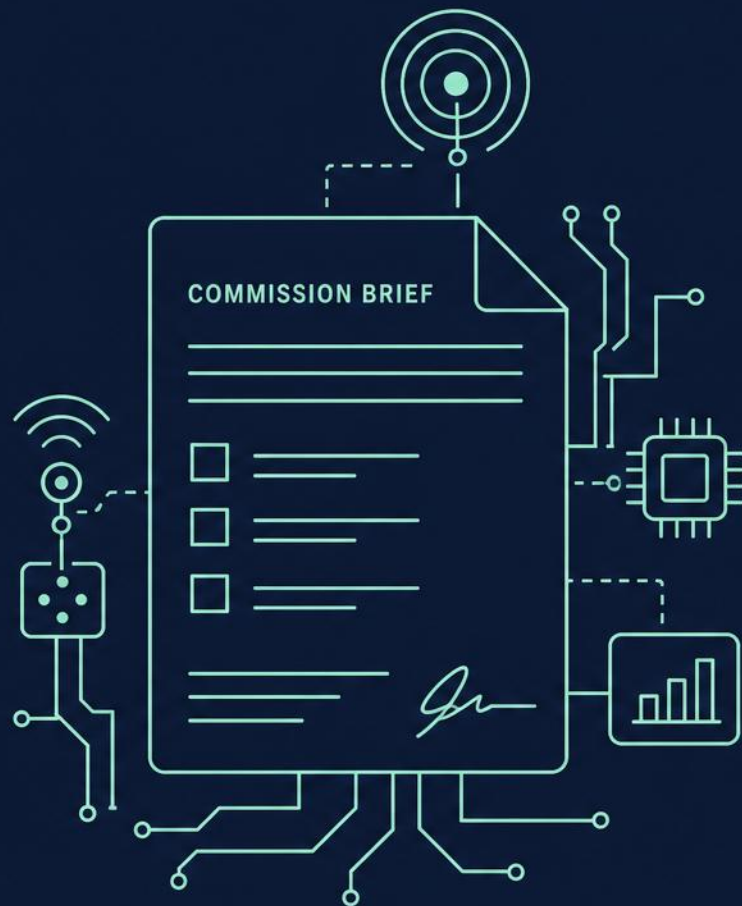


你被任命為架構師



10,000

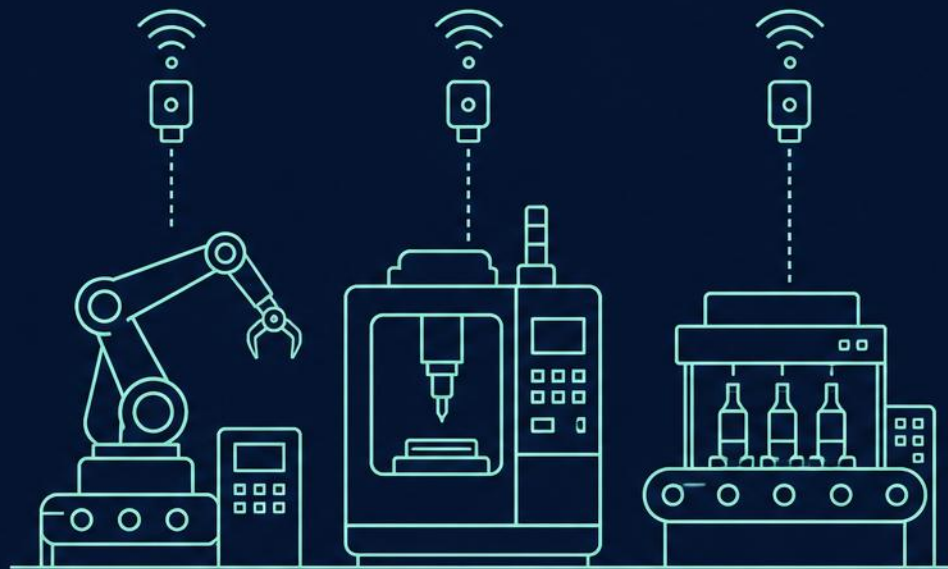
台設備

主角：工廠監控系統

10,000 台設備 · 每 5 秒上報 · 2,000 msg/s 均值 · 尖峰 6,000

告警 P99 < 10 秒；可用性 99.9%

6 人團隊 · 3 個月 · 初期雲費 < \$5,000/月



架構師不是最強工程師

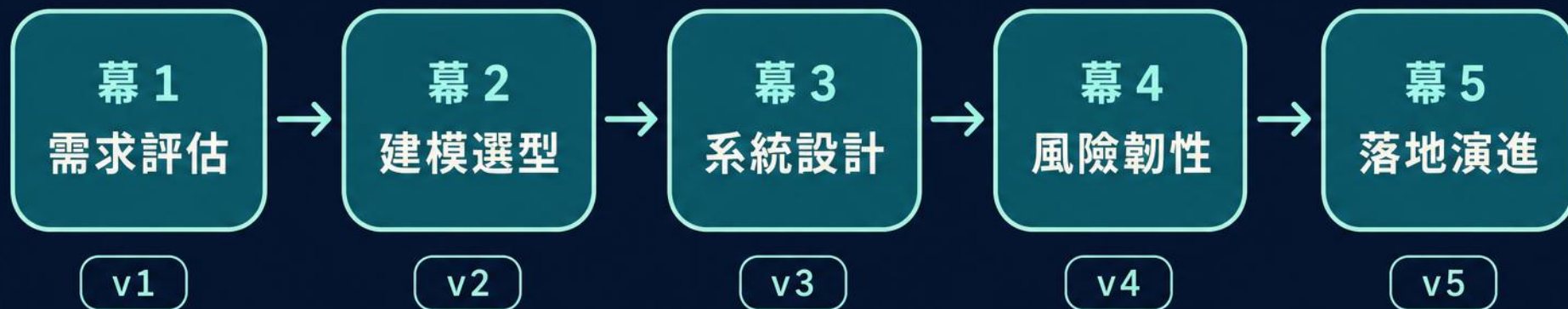
Software Architect / 軟體架構師

把商業目標翻譯成技術決策的人

工程師：讓系統跑起來

架構師：決定系統長成什麼樣

架構師的五幕地圖



五幕走完，白皮書從 v0 長到 v5

每幕一個決策，每決策一張 VCRE 計分

你是決策者，不是旁觀者

架構白皮書 v0 · 接案世界觀

架構白皮書 v0

NEW

你的產出物是一份會長大的設計文件

v0 = 空白委託書，只有需求還沒有答案

每幕結束，白皮書加一個版本

v0

VCRE 計分卡

V

商業價值

Value

這決策賺錢/省錢/降風險嗎？

例：告警快→少賠停機損失

C

成本

Cost TCO

雲費+人力+維運總和？

例：雲費+人力，月控 \$5,000

R

風險

Risk

SPOF 在哪？十倍流量撐得住？

例：TSDB 單點掛 = 全黑

E

可演進

Evolvability

三年後要改，會被卡住嗎？

例：先 monolith，後拆服務

沒有最好的答案，只有取捨。

你的任務開始了

幕一到幕五，你來決定每個架構選擇

結業時能默畫 v1→v5 完整白皮書演化

v1 幕一需求評估



v2 幕二建模選型



v3 幕三系統設計



v4 幕四風險韌性



v5 幕五落地演進

畫給我看

闔上這頁，默畫五幕地圖，v1→v5。

老闆只說要快要穩

「快」是多快？1 秒？10 秒？允許多少例外？

「穩」是幾個 9？99%？99.9%？還是 99.99%？

模糊形容詞無法施工，系統無從驗收。

「快」

「穩」

???

???

把形容詞逼成數字

告警延遲：P99 < 10 秒，允許最多 1% 例外

設備規模：10,000 台，峰值 6,000 msg/s

可用性：99.9%（每月停機 ≤ 43 分鐘）



功能 vs 非功能需求

NFR

非功能需求

系統「怎麼運作」，不是「做什麼」

例：P99 延遲 < 10 秒

SLA

服務等級協議

可用性和延遲的白紙黑字承諾

例：可用性 99.9% 寫進合約

SLO / SLI

SLO 是內部目標，SLI 是量到的實際比例

例：內部 SLO 跑 99.95%

別忘了三種約束

Budget 預算

初期雲費 < \$5,000/月，
超出需重新評估

Deadline 截止日

3 個月 MVP，第 3 個月不能挪

Team Skills 技能樹

6 人熟 Python，剛接觸雲端

架構白皮書 v1 · 需求與約束

白皮書 v1：需求書

PRD 功能清單 ✓ · NFR 矩陣 ✓ · 約束清單 ✓

指標	目標值	量測方式
告警延遲	P99 < 10 秒	Consumer 到 Webhook 時間差 P99
峰值吞吐	6,000 msg/s	負載測試 5 分鐘峰值
可用性	99.9%	月度 Uptime 監控
每日資料量	≤ 35 GB/天	DB chunk 每日大小
初期雲費	< \$5,000/月	月帳單 Cost Explorer
停機損失	~\$20,000/hr	業務合約基準

白皮書 v1

Budget \$5,000/月

Deadline 3 個月 MVP

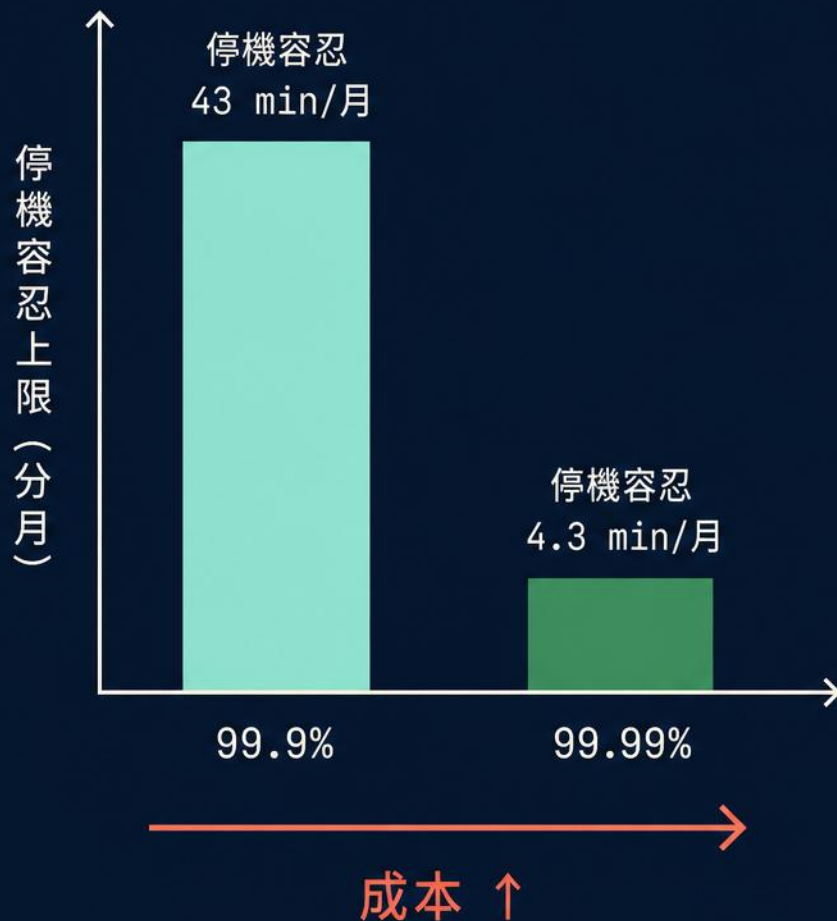
Skills 6 人 Python+Cloud

多一個 9 貴十倍

99.9% → 99.99% : 停機容忍從 43 分/月 → 4.3 分/月

做到 4.3 分需熱備援、自動切換、多 AZ

雲費與複雜度通常跟著翻倍以上



該追求 99.99% 嗎

99.99% 可用性意味著一年僅約 52 分鐘停機時間，對使用者體驗與品牌信任有顯著助益。

但達成需投入更多成本與複雜度：多 AZ、熱備援、自動化故障切換與更嚴謹的監控。

在資源有限的情況下，我們需要用 VCRE 框架評估，找到當下最平衡的選擇。

V

商業價值

2/5

MVP 規模未到，收益有限

C

成本

4/5

熱備援 + 多 AZ，雲費月增高

R

風險

4/5

複雜度翻倍，新手團隊難駕馭

E

可演進

5/5

先穩 99.9%、v4 再演進

結論：v1 定 99.9%，v4 演進至 99.99%。

SCENARIO

名詞滿天飛

工程師叫「data point」、PM 叫「讀數」、DBA 叫「row」

同一件事叫三個名字，需求就會漏掉

建模第一步：把名詞統一成一張表

data point

讀數

row

核心名詞與邊界

哪些是核心實體？誰擁有資料？

Device 跟 Sensor 是同一件事嗎？

Alert 是資料還是事件？邊界在哪？



實體？



所有權？



語義？

統一語言與限界

Ubiquitous Language / 統一語言

— 全團隊用同一套術語描述業務

例：全員都叫它 Reading

Bounded Context / 限界上下文

— 術語意義有效的邊界範圍

例：監控與計費各自獨立

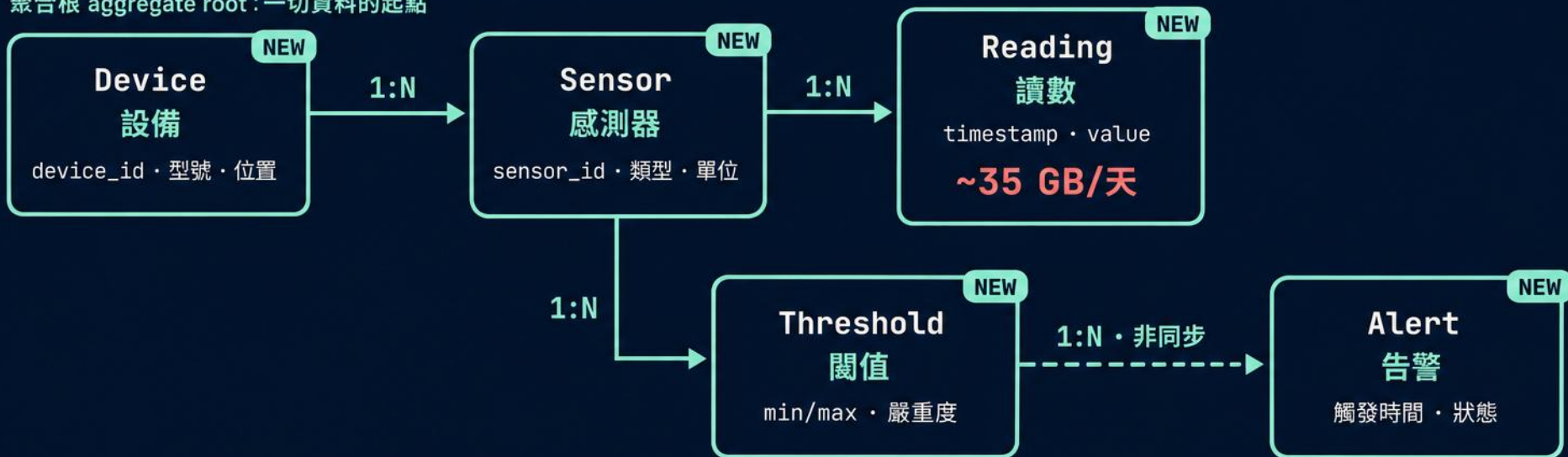
Domain Model / 領域模型

— 業務實體與關係的結構化描述

例：Device 一對多 Reading

白皮書 v2：領域模型

聚合根 aggregate root：一切資料的起點



1:N = 一對多 虛線 = 非同步（事後比對才觸發）

白皮書 v2

一台設備有多個感測器，每 5 秒產生讀數
感測器設閾值，超標就觸發告警

時序資料怎麼存

每天 **35 GB** 讀數，SQL 還是 NoSQL？

查詢：過去 **1 小時** 某感測器平均值

刪除：超過 **90 天** 資料自動 purge

35 GB / 天



SQL vs NoSQL

ACID / ACID 事務

— 強一致、原子、可回滾的事務保障

例：寫入讀數要嘛全成全敗

BASE / BASE 特性

— 最終一致、高可用、犧牲即時一致

例：告警延幾秒可接受

Time-Series DB / 時序資料庫

— 以時間為主鍵、原生壓縮與保留策略

例：35 GB/天 按時間查

白皮書 v2: 技術棧

層級	技術選擇	說明
API 後端	FastAPI (Python)	熟悉語言、async 支援
時序儲存	PostgreSQL + TimescaleDB	SQL 介面 + 時序壓縮
快取層	Redis	P99 < 10s SLA 輔助
訊息佇列	Kafka	尖峰 6,000 msg/s 緩衝

白皮書 v2

Hypertable = TimescaleDB 按時間自動切塊的表
按時間切塊 = 加速時間查詢

ADR-001 · 選用 TimescaleDB 作時序儲存

Context: 每天 35 GB 讀數，主要查詢為時間範圍聚合，需 90 天 retention policy

Decision: 以 TimescaleDB extension 擴充既有 PostgreSQL，不引入全新系統

Consequences: ⊕ 保留 SQL 查詢能力；⊕ 原生壓縮降低儲存費；⊕ 自動 retention；
⊖ extension 版本需與 PostgreSQL 對齊



PostgreSQL



TimescaleDB



FastAPI



redis

時序 DB 值得嗎

時序資料量大且持續成長，查詢與儲存成本是關鍵挑戰。

TimescaleDB 以 PostgreSQL extension 方式提供原生時序能力。

我們用 VCRE 框架評估，確認它是否值得導入。

V 商業價值
4/5

壓縮 35→3.5GB，符合預算與 SLA

C 成本
3/5

免費 extension，維護成本可控

R 風險
2/5

同生態 extension，不引入新基建

E 可演進
4/5

可加 Replica，升 Citus 不搬家

結論：TimescaleDB 是 PostgreSQL extension，不是新系統——值得。

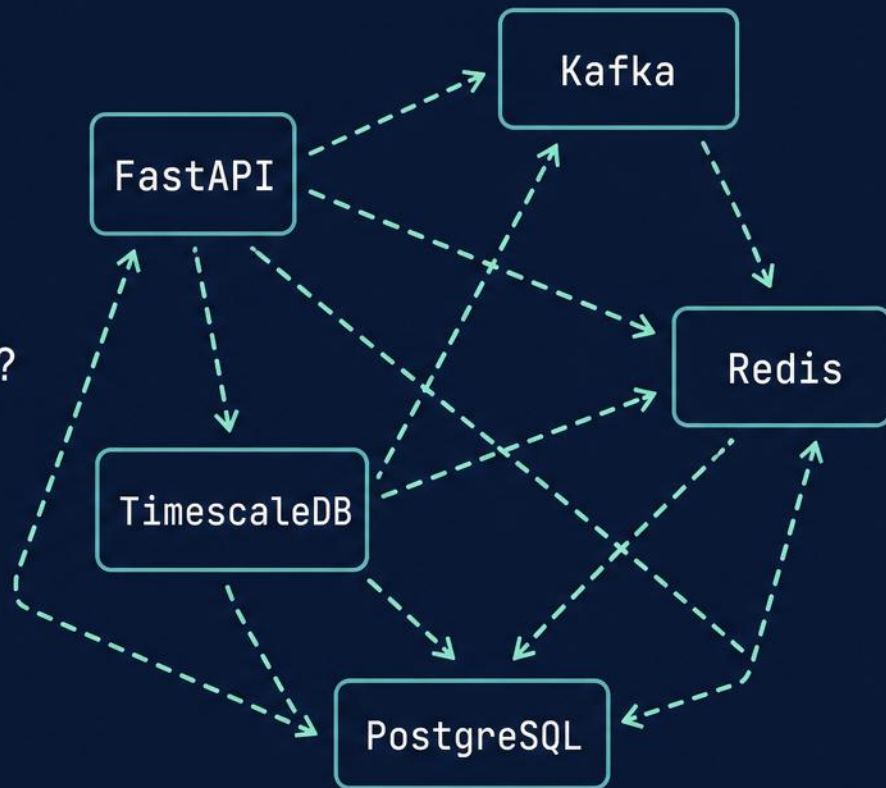
SCENARIO

元件兜不起來

有模型、有技術棧，元件怎麼「接線」還是糝糊

FastAPI、Kafka、Redis——誰先誰後、誰呼叫誰？

不畫圖，實作時才發現接錯就來不及了



KEY QUESTIONS

要微服務嗎

通訊用 REST、gRPC 還是 Queue？語意不同，保證不同

哪些服務要 stateless？Session 和狀態放哪裡？

上報路徑和查詢路徑能共用同一個 API 嗎？



C4 四層視角

Context / 系統情境

系統與外部使用者、外部系統的邊界與互動

例：設備與儀表板誰在用

Container / 可部署單元

可獨立部署的服務、資料庫、佇列、前端

例：API、Kafka、DB 各一塊

Component / 模組

Container 內部的主要模組或服務邏輯單元

例：Query API 內的查詢模組

Code / 類別與函數

Component 內的類別、介面、實作細節

例：查詢函式與資料類別

白皮書 v3 停在 Container 層

為何分層：先看整體再看細部

對不同對象講不同層

削峰用佇列

Stateless / 無狀態服務

服務不保存請求間狀態，可任意水平擴展

Cache / 快取

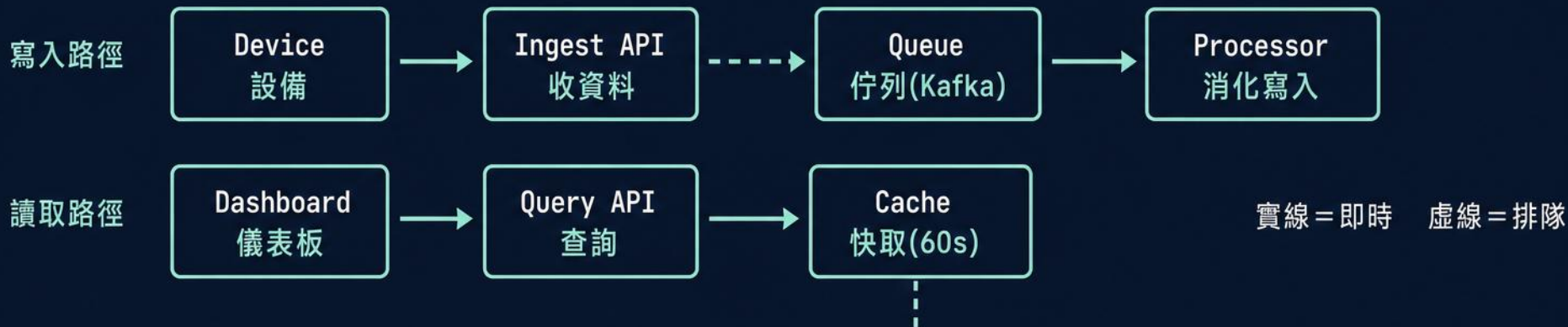
將熱點查詢結果暫存，避免重複打資料庫
cache-aside：先查快取，miss 再打 DB 並回填

Message Queue / 訊息佇列

生產者與消費者解耦，吸收尖峰流量

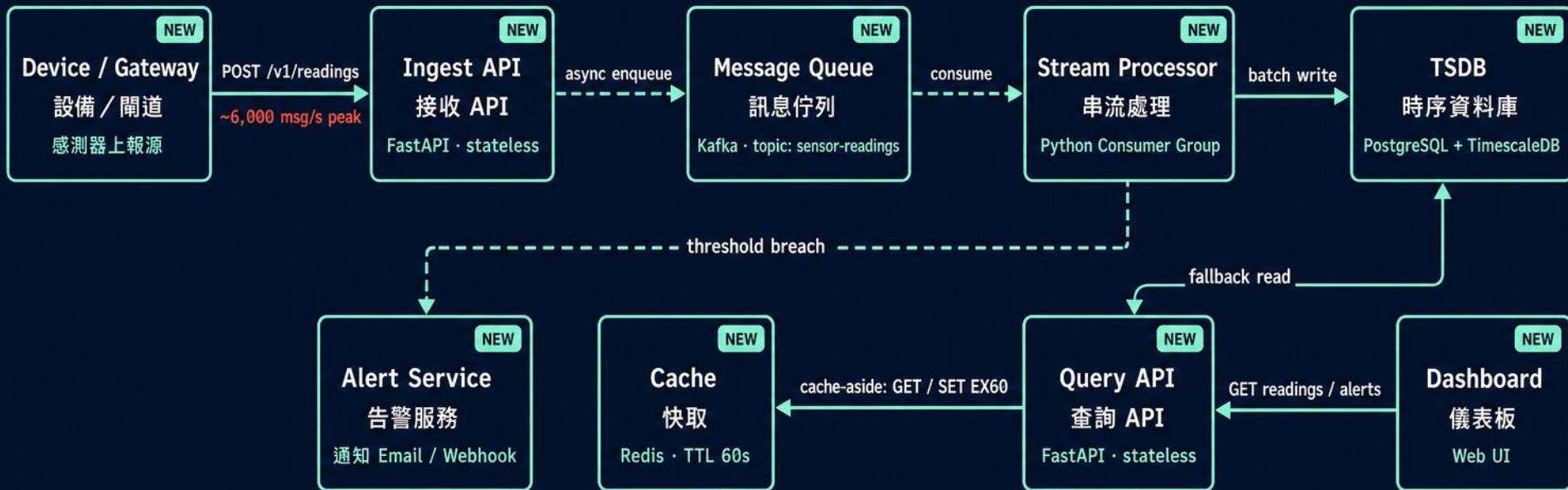
同步寫：尖峰 6,000 msg/s 直接砸 DB 會飽和

佇列削峰：先進 Kafka，Consumer 慢慢消化



白皮書 v3 : C4 容器圖

—— 實線 = 同步
----- 虛線 = 非同步



C4 容器層：每個方塊 = 一個可獨立部署的服務

白皮書 v3



FastAPI



kafka



redis

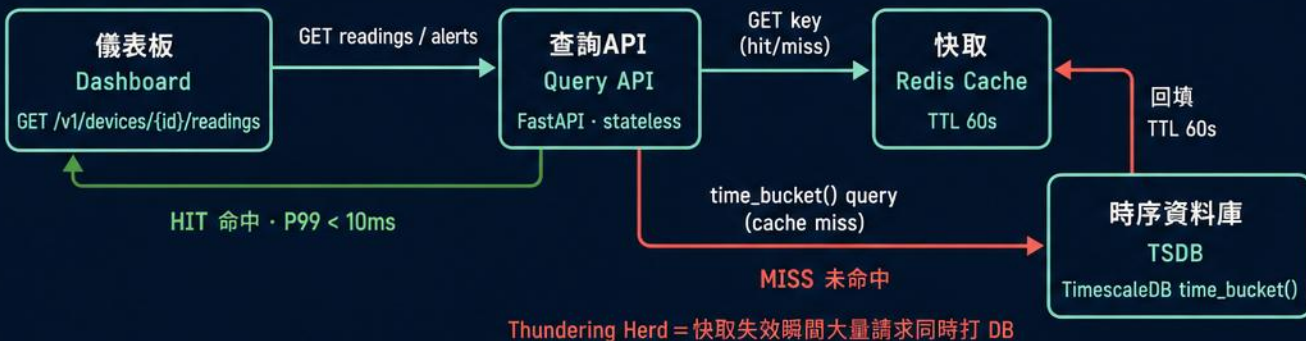
白皮書 v3：關鍵資料流

實線 = 同步 虛線 = 非同步

WRITE PATH 寫入路徑



READ PATH 讀取路徑



ALERT PATH 告警路徑



API 草稿

```
POST /v1/readings
GET /v1/devices/{id}/readings
GET /v1/devices/{id}/alerts
```



Kafka



Redis



FastAPI



PostgreSQL

白皮書 v3

同步還是佇列

目前同步寫入 **TSDB** 在尖峰會打到 **6,000 msg/s**，導致延遲與失敗風險上升。

改用 **Kafka** 佇列可先行緩衝，平滑流量並解耦 **Ingest** 與後端處理。

後續由 **Consumer** 批次拉取寫入 **TSDB**，兼顧可用性與擴充性。

V 商業價值
4/5

削峰保護 TSDB，護住可用性

C 成本
3/5

多一層 Kafka，學習成本明顯

R 風險
4/5

Kafka 成新依賴，SPOF 候選

E 可演進
5/5

加 Consumer 即擴，Ingest 不動

結論：Queue 非同步是正確選擇，成本是多學一層 Kafka。

SCENARIO

TSDB 掛了就瞎了

上線前一晚：TimescaleDB 單實例掛掉

告警停了、讀數查不到、整廠監控全黑

停機一小時 = \$20,000 的代價

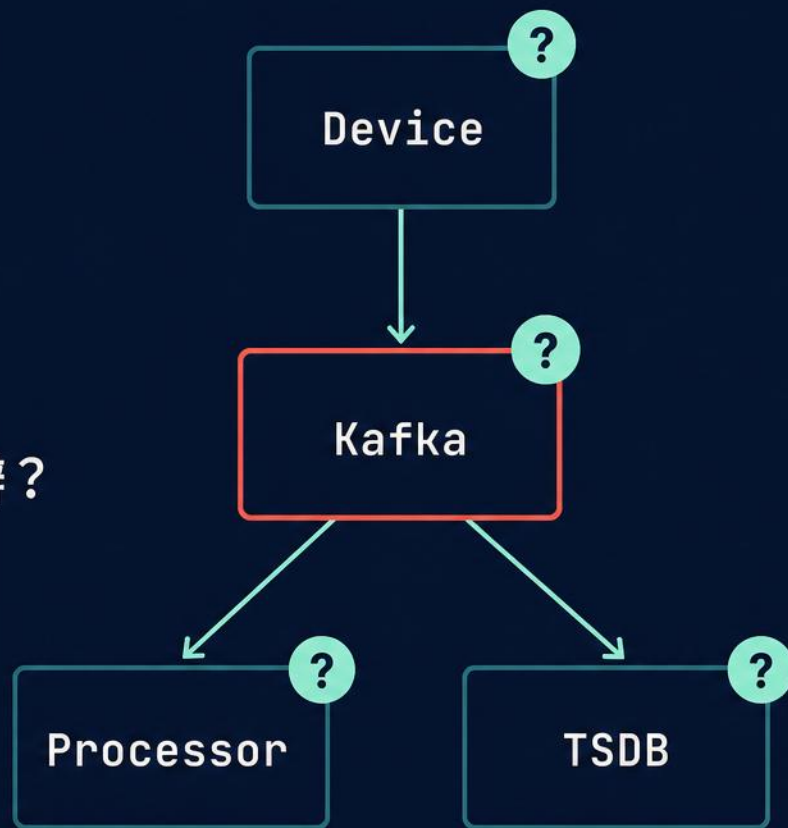


SPOF 在哪

流量暴增 10 倍，哪個元件先垮？

Kafka 只有一個 broker，queue 塞爆怎辦？

哪個元件死了讓告警 P99 破 10 秒？



找出單點故障

SPOF

一個元件掛掉即造成整體服務中斷

例：單台 TSDB 掛→全廠黑

Availability

系統在約定時間內正常服務的比例

例：99.9% = 每月可停 43 分

FMEA

逐一問：這個元件壞了，影響是什麼？

例：逐一系列元件失效與影響

三術語的具體例子

SPOF：單台 TSDB

單台 TSDB 掛掉會怎樣？

寫入停・讀取停・告警靜默

→ 全廠監控全黑 \$20,000/hr

Availability：99.9% 是多少

一年 8,760 小時 $\times$ 0.1%

$\approx$ 8.76 小時/年 $\approx$ 43 分/月

99.99% 只剩 4.3 分/月 (多一個9)

FMEA：一系列示範

元件：TSDB 單台

失效：全廠監控黑、\$20k/hr

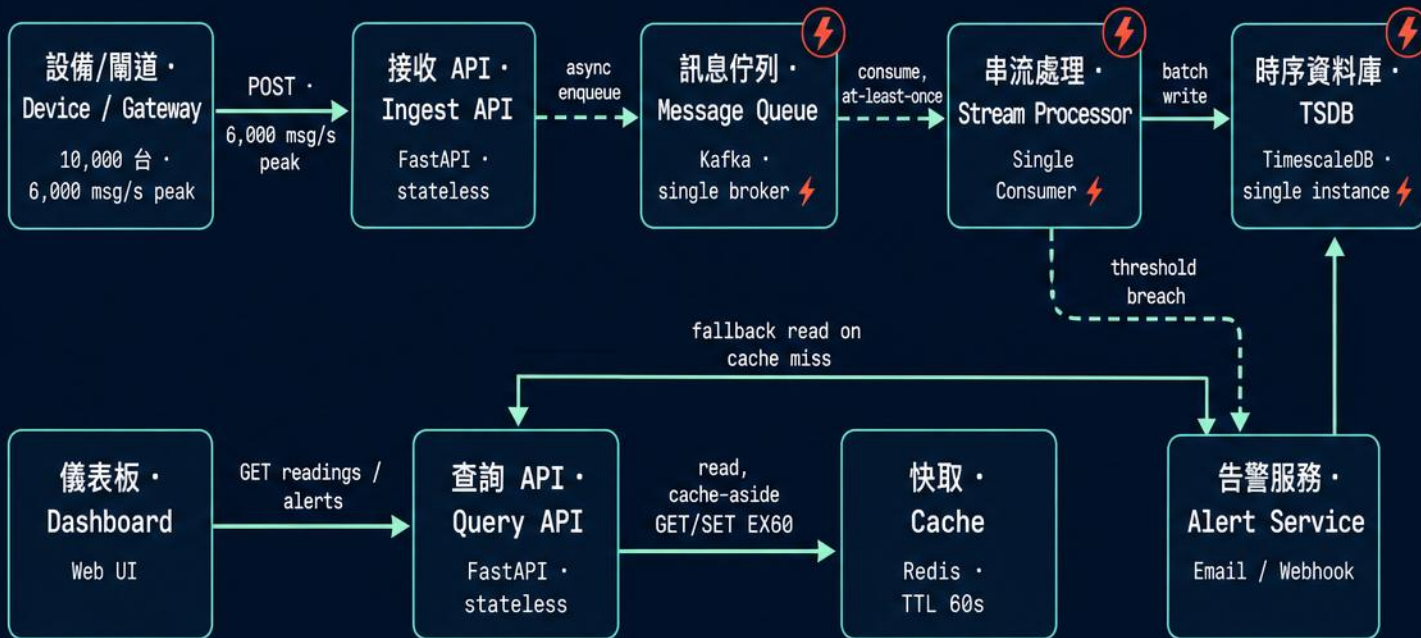
緩解：加 Replica 主備+讀寫分離

定義記不住？看一個例子就懂了

五種韌性手法

術語	一句定義
Replica / 複本	資料與服務的多個副本，任一掛掉可切換 例：TSDB 主備，備庫接讀 Streaming Replication = 主庫即時把變更串流到備庫
Retry / 重試	失敗後自動再試，搭配退避避免雪崩 例：寫失敗自動重試 Backoff = 重試間隔指數遞增，避免雪上加霜
Idempotency / 冪等	相同請求重複執行，結果一致，Retry 安全 例：同筆重送只記一次
Backpressure / 背壓	下游告知上游放慢，避免佇列無限膨脹 例：Consumer 落後就限流
Circuit Breaker / 熔斷	偵測下游異常後斷開，防止故障擴散 例：下游異常先斷開

白皮書 v4 : 故障模式圖



⚡ = 單點故障 SPOF 實線 = 同步 虛線 = 非同步

白皮書 v4

FMEA 摘要

- ⚡ TSDB 單實例 ·
影響：讀寫全斷 \$20k/hr ·
緩解：Replica + 讀寫分離
- ⚡ Kafka 單 broker ·
影響：上報寫入中斷 ·
緩解：複本 + 背壓
- ⚡ Stream Processor ·
影響：告警 P99 破 10s ·
緩解：Consumer Group 擴展 + 重試/冪等



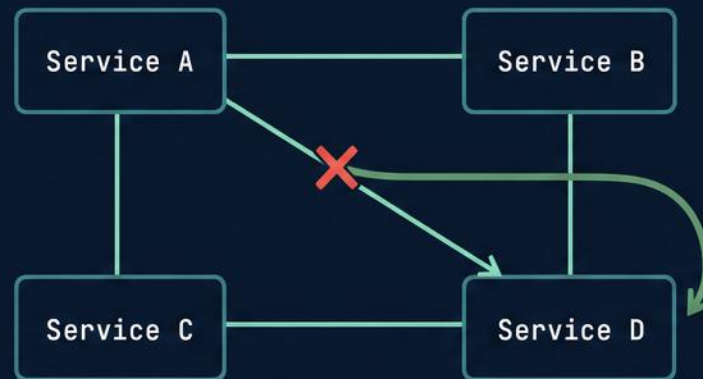
大廠主動弄壞自己

混沌工程：對三個 SPOF 注入故障，驗證自動切換

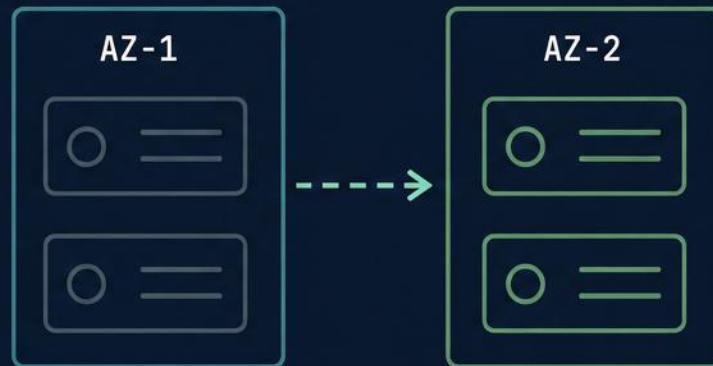
多 AZ 部署：跨可用區，單 AZ 掛不影響整體

業界共識：韌性要演練，不能只靠設計

Chaos



Multi-AZ



多 AZ 備援值得嗎

多 AZ 備援能提升服務可用性與容錯能力，降低單一可用區域故障風險。

多 AZ 雲費額外增加 \$3,000-6,000/月，對中小團隊是明顯負擔。

以目前 6 人團隊與成長階段，應審慎評估投資時機與替代方案。

V

商業價值

3/5

提升可用性，但收益假設性

C

成本

5/5

跨 AZ 複製，總費超 \$5k 預算

R

風險

3/5

跨 AZ 複雜，新手團隊風險中高

E

可演進

5/5

v4 先緩解 SPOF，v5 再多 AZ

結論：v4 先做 SPOF 緩解，v5 再演進多 AZ。



工程師不知道幹嘛

拿到架構圖，還是問「今天 commit 什麼」

上線後沒監控，靠用戶回報才知系統壞了

架構師沒做完的最後兩件事





開工



監控



演進

怎麼讓人開工

怎麼開工？分支怎麼開？CI 守什麼門？

上線後怎麼知道系統活著？

何時才該把 monolith 拆成微服務？

開發護欄

GitHub Flow / 分支策略

6 人推薦，短命分支合 PR

例：feature 分支 + PR

Linters / Formatter / 格式自動化

ruff + black ; commit 前自動執行

例：ruff+black commit 前跑

CI Gate / 合併守門

PR 合 main 前必過 lint + test

例：測試沒過不准合

Scaffold / 專案腳手架

新成員 5 分鐘能跑起來的 make init

例：make init 一鍵起服務

可觀察性三本柱

Logs / 事件記錄

結構化 JSON；記錄是什麼事件發生了

例：記下告警觸發事件

Metrics / 可聚合數值

CPU、latency、lag；看趨勢與告警閾值

例：CPU、consumer_lag

Traces / 跨服務請求路徑

端對端追蹤；找出慢在哪個服務

例：一次請求跨服務追蹤

OpenTelemetry / 統一遙測 SDK

一次接入，同時輸出三類訊號

例：一套 SDK 收三類訊號

為何用 OTel 統一 SDK：
一次接入三類訊號，免維護三套庫

白皮書 v5：開發規範

專案結構

```
iot-monitor/  
├──  ingest/      # 上報接收  
├──  processor/    # 消費 + 告警  
├──  query/        # Dashboard API  
├──  shared/       # 共用 model/config  
├──  tests/          
├── .pre-commit-config.yaml  
└── pyproject.toml
```

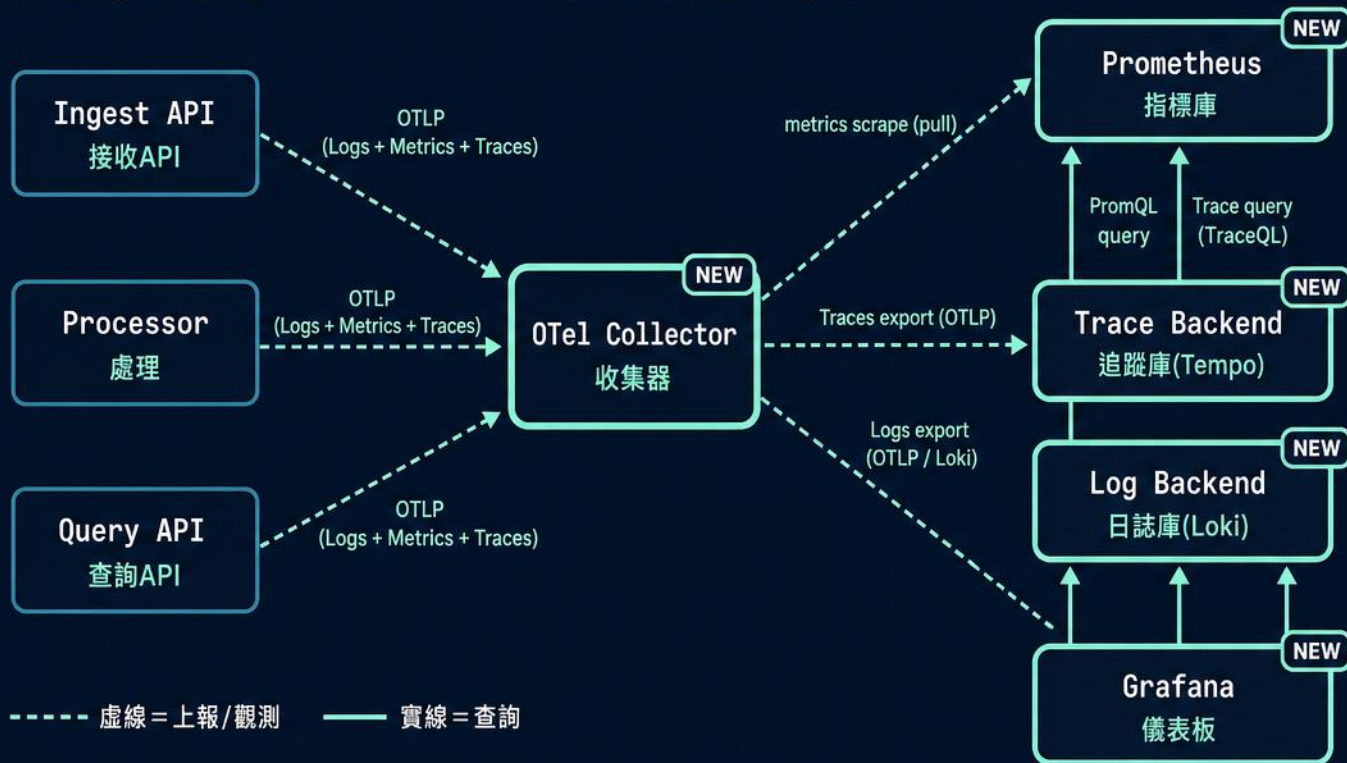
開發規範

分支：GitHub Flow, feature/<name>, PR 合 main

Lint: ruff + black, pre-commit hook 自動執行

CI: PR 合 main 前必過 lint + unit test

白皮書 v5：可觀察性



白皮書 v5

演進路線

● 現在 monolith

● 拆微服務條件：

- 團隊具備領域建模與 DevOps 能力
- 部署與發版已自動化與可觀測
- 資料一致性與分散式治理就緒

● 更遠 Event Sourcing/CQRS

Event Sourcing = 存每次變更事件，可回溯歷史
CQRS = 讀寫模型分離，各自最佳化



OpenTelemetry



Prometheus



Grafana

說服老闆出錢

無實權影響力

/ Influence Without Authority

沒有預算決定權，靠論點改變決策

對老闆說：

“停機 \$20,000/小時；監控 \$300/月；
故障發現從 30 分鐘→1 分鐘，
省 > \$10,000/次”

因人而異溝通

/ Audience-Adapted Framing

對老闆講錢；對工程師講時間

對工程師說：

“有 Traces，15 分鐘找到慢查詢；
沒有，你除錯三天”

一開始就微服務嗎

1. 微服務不是銀彈，初期會放大複雜度與組織成本。
2. 先以最快速度驗證商業假設與市場反應。
3. 依據明確的觸發條件再拆分，讓拆分有價值。

V

商業價值

4/5

monolith 讓 3 個月 MVP 達標

C

成本

2/5

免 K8s / Service Mesh，雲費可控

R

風險

2/5

無分散式事務與跨服務風險

E

可演進

4/5

package 邊界預留，可沿邊界拆

結論：modular monolith 先行；有觸發條件再拆。

RECAP

v0 → v5

架構白皮書演化歷程

五幕走完了

從空白到可落地，架構白皮書逐步成形。

每一版都是決策與共識的累積。

這份文件，記錄了我們一起走過的路。

v0 空白委託書

v1 初版草案與目標對齊

v2 架構藍圖與技術選型

v3 整合規劃與風險管理

v4 落地方案與執行計畫

v5 最終定稿與交付就緒

白皮書演化 v1→v5

每一版在上一版加一個能力層

v1 需求與約束 · 量化需求 NFR

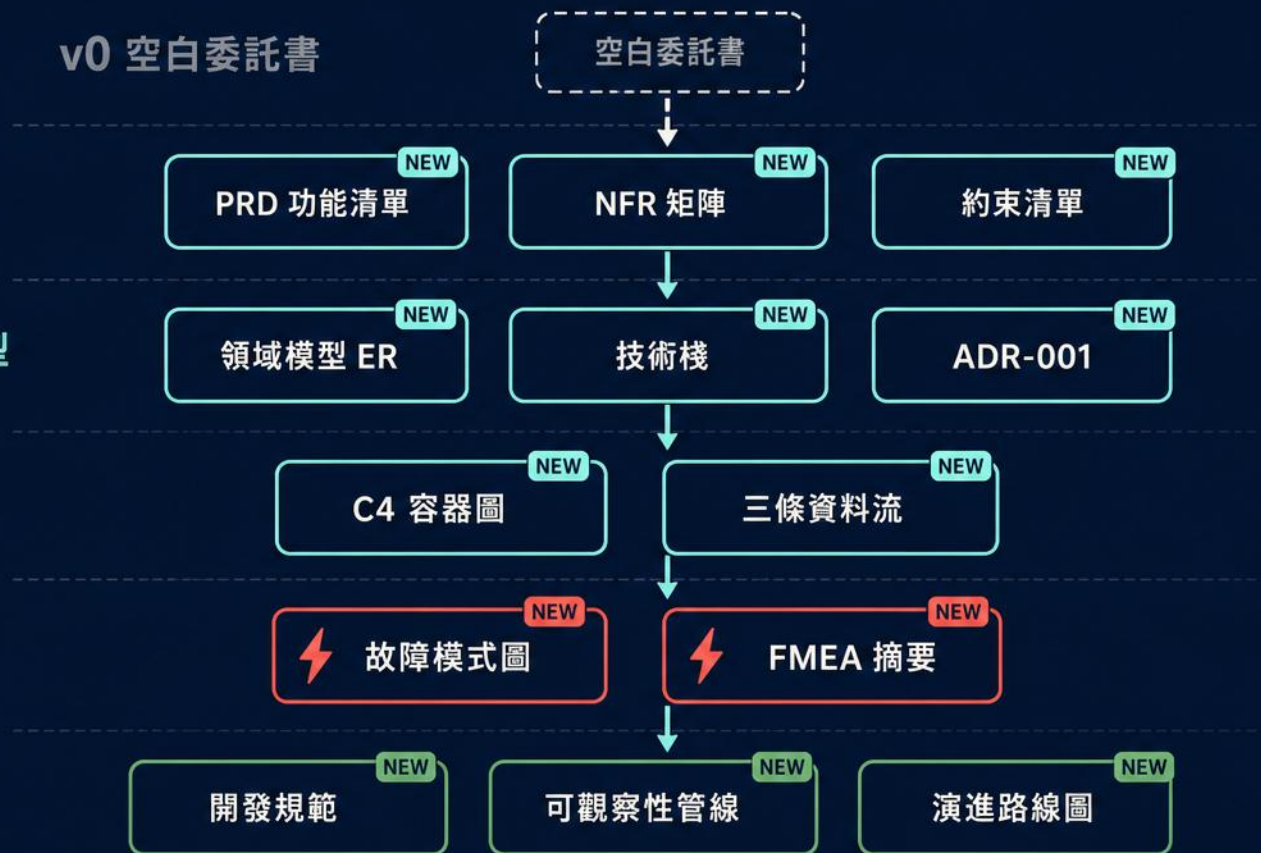
v2 建模與選型 · 領域模型 + 選型

v3 系統設計 · 畫出系統全貌

v4 風險與韌性 · 標風險 + 緩解

v5 落地與演進 · 上線 + 可演進

v0 空白委託書



v1 → v5

一頁式架構白皮書

v1→v5

系統名稱

IoT 設備監控系統（即時告警）

關鍵 NFR

告警 P99 < 10s · 峰值 6,000 msg/s · 可用性 99.9%
每日資料量 ≤ 35 GB · 雲費 < \$5,000/月

技術棧

FastAPI · PostgreSQL+TimescaleDB · Redis · Kafka

核心架構

modular monolith · Ingest/Processor/Query 三包
非同步削峰（Kafka）· 快取輔助（Redis · TTL 60s）

主要風險與緩解

TSDB SPoF → Replica + 讀寫分離
Kafka 單點 → 3-broker cluster
Processor 單點 → Consumer Group ≥2 + Idempotency

演進觸發條件

團隊 > 20 人 / 獨立擴展需求 → 沿 package 邊界拆微服務
讀取負載 > 寫入 5x → CQRS（屆時再評）

換你默畫一次

不看講義，默畫 v1→v5 + 每步關鍵決策

畫出：哪個版本加了什麼、為什麼加

完成後對照評分標準，看你遺漏了哪一步

v1 · 需求與約束

- ☐ 列出 NFR 矩陣 (P99 < 10s / 6,000 msg/s / 99.9%)
- ☐ 說出三個約束 (\$5,000/月 / 3 個月 / 6 人)

v2 · 建模與選型

- ☐ 畫出五個領域實體 (Device / Sensor / Reading / Threshold / Alert)
- ☐ 說出選 TimescaleDB 的原因 (壓縮 / retention / SQL 不變)
- ☐ 知道 Kafka 是為了什麼 (尖峰 6,000 msg/s 削峰)

v3 · 系統設計

- ☐ 畫出三條路徑 (寫入 / 讀取 / 告警)
- ☐ 說出 Ingest API 為何回 202 (非同步解耦)
- ☐ 說出 cache-aside 的 hit/miss 流程

v4 · 風險與韌性

- ☐ 說出三個 SPOF (TSDB / Kafka / Processor)
- ☐ 說出 TSDB 的緩解手法 (Replica + 讀寫分離)
- ☐ 說出 Kafka 緩解 (3-broker cluster · replication.factor=3)
- ☐ Stream Processor : Consumer Group ≥ 2 + 冪等

v5 · 落地與演進

- ☐ 說出可觀察性三本柱 (Metrics / Traces / Logs)
- ☐ 說出拆微服務的觸發條件 (> 20 人 / 獨立擴展需求)

VCRE 決策總複盤

把一路的關鍵選擇，用同一把尺（VCRE）量一次。

每一幕都是在有限資源下，選一條能活下來、能長大的路。

接下來，我們照這張表的結論推進下一版的設計與驗證。

幕	決策題目	V	C	R	E	結論
幕 1	99.9% vs 99.99% 可用性	2	4	4	5	v1 定 99.9%，v4 演進至 99.99%
幕 2	TimescaleDB vs 通用 PostgreSQL	4	3	2	4	選 TimescaleDB (extension，非新系統)
幕 3	同步寫 vs Queue 非同步 (Kafka)	4	3	4	5	Queue 非同步，成本是多學一層 Kafka
幕 4	多 AZ 現在做 vs 等 v5	3	5	3	5	v4 先做 SPOF 緩解，v5 再多 AZ
幕 5	微服務先行 vs Modular Monolith	4	2	2	4	Modular monolith 先行，有觸發條件再拆

V/E 高=好；C/R 高=該擔心（幕 3 R=4 = Kafka 新依賴；幕 4 C=5 = 超預算）；幕 5 C/R 低=成本低/風險低，是優點

你能當架構師了

從 v0 到 v5，你做了真實的架構決策

下一步：分散式系統深化、真實專案實作

面試時：說決策理由、說取捨、說演進路徑

01

深化分散式系統

CAP 定理 · 一致性模型 · 分散式事務 · 服務網格

02

做一次真實專案

挑一個 GitHub repo 或線上服務，用 v1 需求書起手
從需求到 FMEA，走一遍五幕

03

面試時的架構決策表達

不背圖——說 Why：需求驅動 → 選型理由 →
VCRE 取捨 → 演進路徑

v0 → v5 完成了。架構師的旅程從這裡繼續。